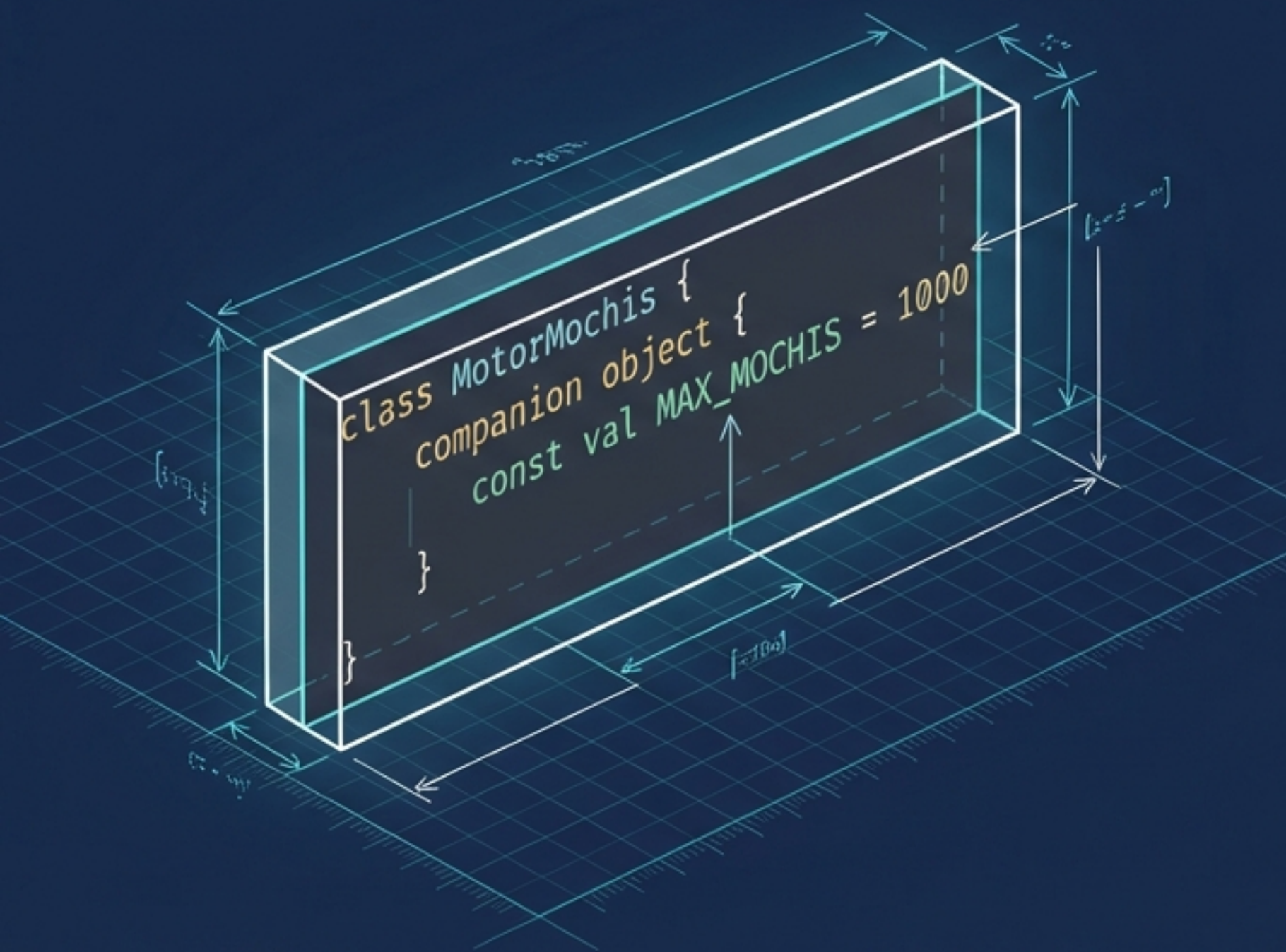


Del Plano al Lienzo: Anatomía de una App en Jetpack Compose

Cómo transformar 4 piezas de código Kotlin en un motor físico interactivo.



Un enfoque paso a paso
para dominar Estado,
Lógica y UI en Android.

Toda la Magia Ocurre en 4 Bloques



1. El Dato (El Qué)
La clase Mochi



2. El Motor (El Cómo)
MotorMochis



3. El Puente (El Dónde)
MainActivity



4. El Lienzo (La Interfaz)
@Composable PantallaMochis

Si entiendes estas 4 piezas, entiendes el desarrollo moderno en Android.

Pieza 1: El Dato (Construyendo el ADN del Mochi)

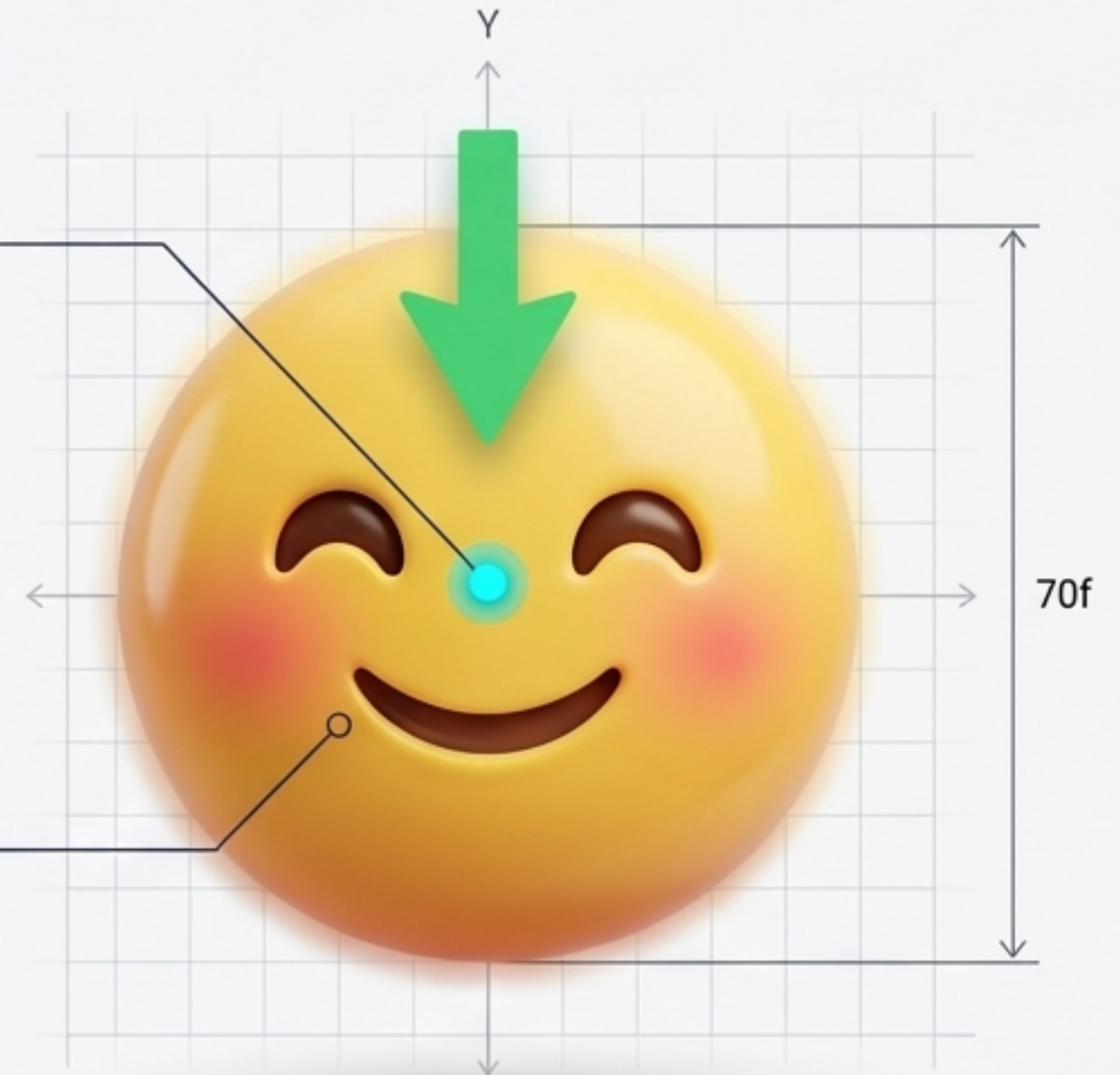
```
data class Mochi(  
    var x: Float, var y: Float,  
    var velocidadY: Float = 0f,  
    var velocidadX: Float = 0f,  
    val radio: Float = 70f,  
    var emoji: String,  
    val tiempoCreacion: Long  
)
```

Posición exacta en la pantalla

Fuerza de gravedad cayendo

Tamaño fijo de 70 píxeles

El carácter visual



val vs. var: La Regla de la Mutabilidad

var (Mutable / Abierto)



```
var velocidadY: Float
```

Puede cambiar y reescribirse constantemente.

La velocidad del Mochi cambia en cada fotograma mientras cae y rebota. Necesitamos que sea variable.

val (Immutable / Cerrado)

```
val tiempoCreacion: Long =  
    System.currentTimeMillis()
```



Una vez asignado, es permanente.
No se puede alterar.

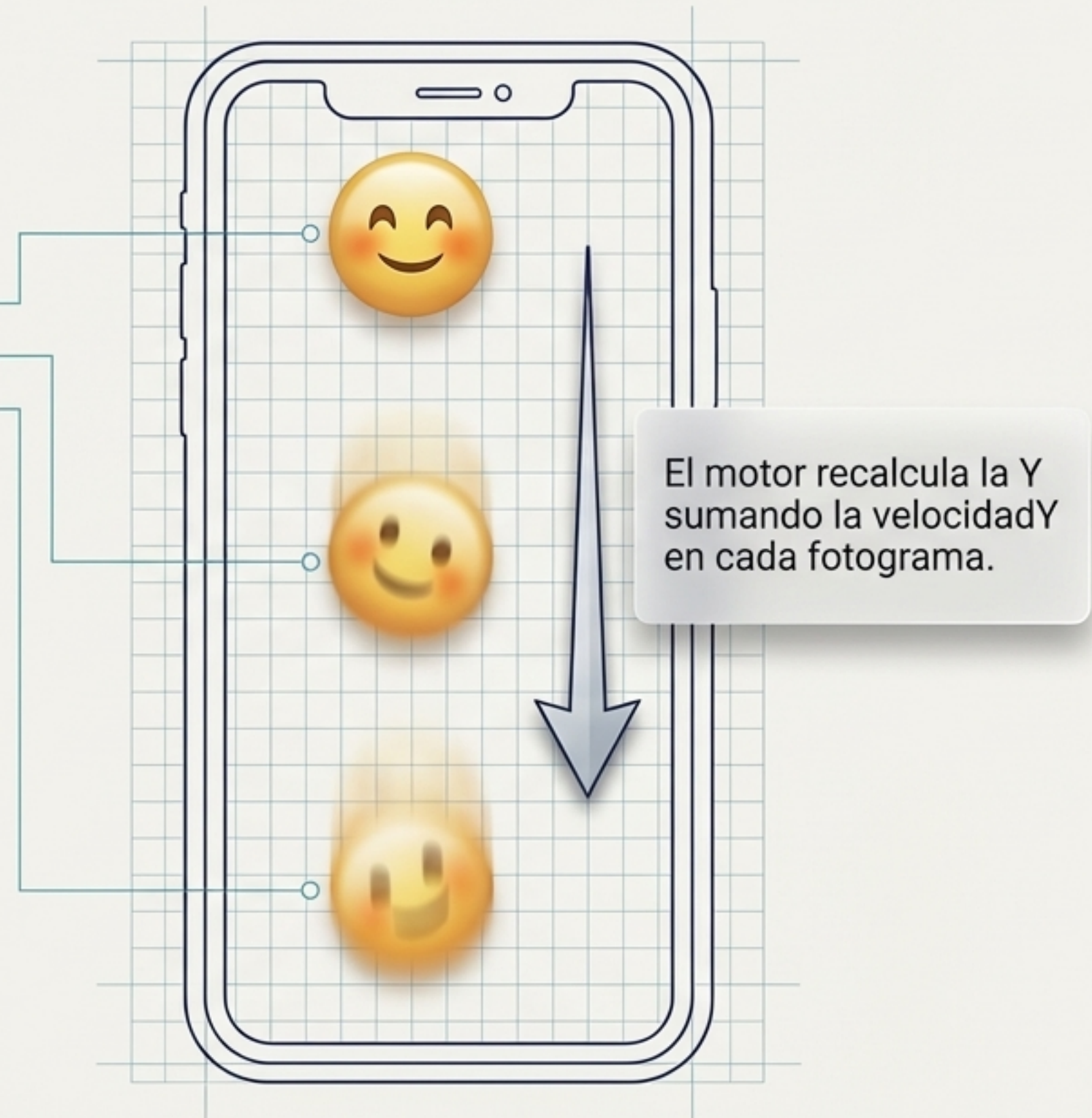
El momento exacto en que nace el Mochi nunca cambiará en el futuro. Es un hecho immutable.

Pieza 2: El Motor (Las Leyes de la Física)

```
class MotorMochis {  
    companion object {  
        const val MAX_MOCHIS = 1000  
    }  
}
```



Para evitar que el teléfono colapse, el motor establece un límite estricto: **MAX_MOCHIS = 1000**. Si creamos el 1001, el más antiguo desaparece.



Pieza 3: El Puente (Cruzando hacia Compose)

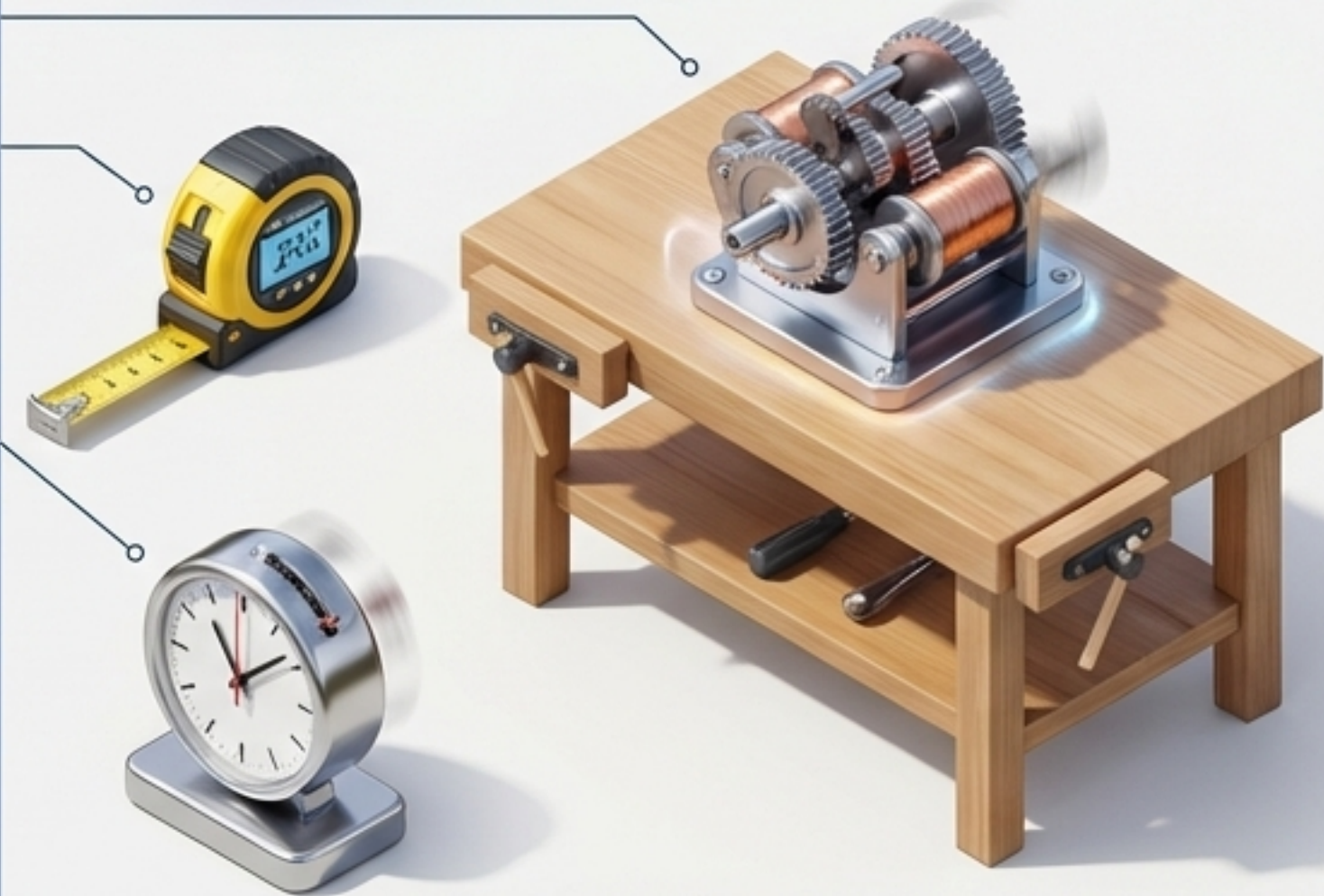


Pieza 4: El Lienzo (Preparando los Materiales)

```
@Composable  
fun PantallaMochis() {  
  
    val motor = remember { MotorMochis() }  
  
    var tamanyoPantalla by remember  
        { mutableStateOf(IntSize.Zero) }  
  
    var contadorFotogramas by remember  
        { mutableStateOf(0) }  
  
}
```

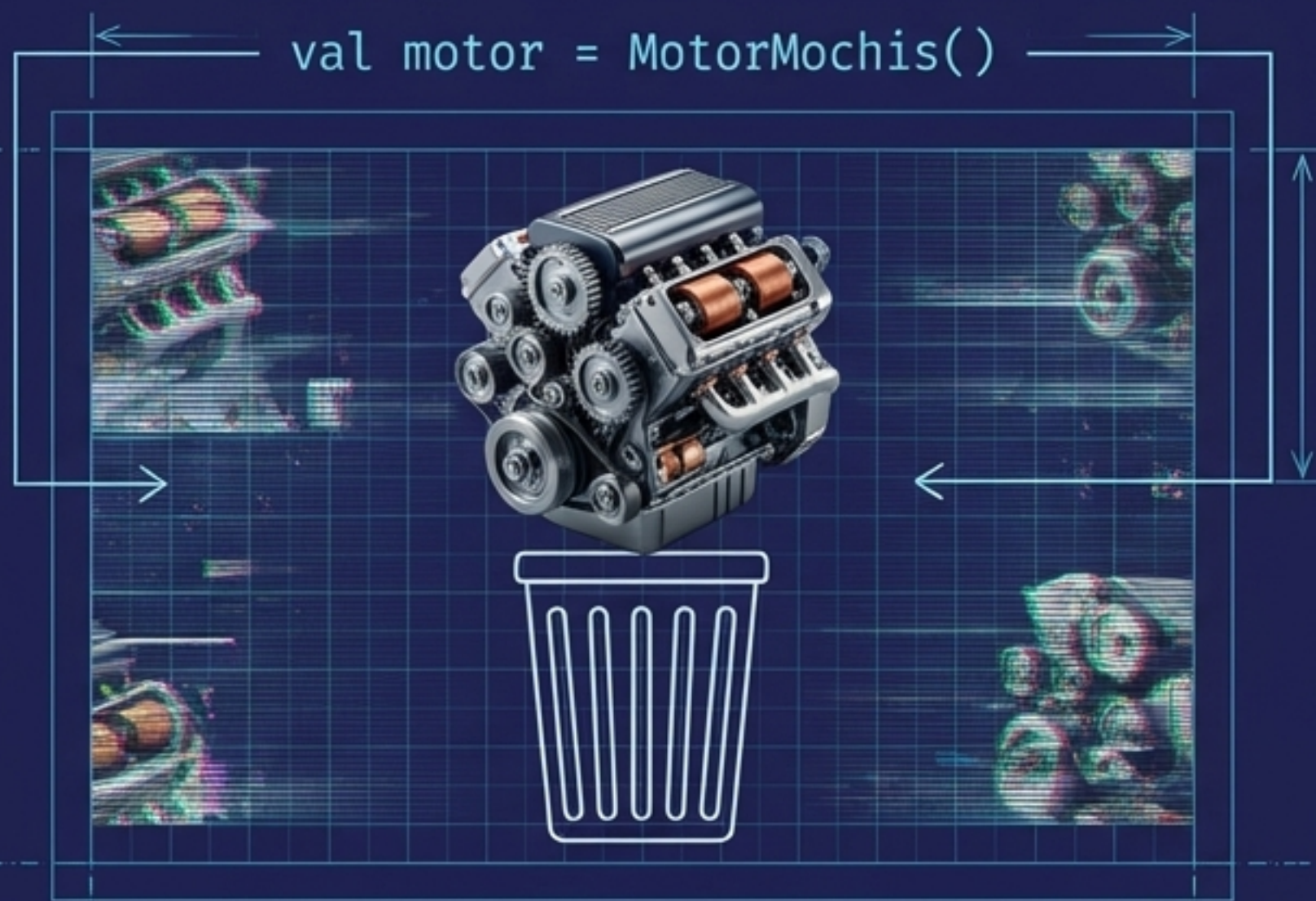


Esta etiqueta le dice a Kotlin que esta función no devuelve datos, devuelve Interfaz Gráfica.

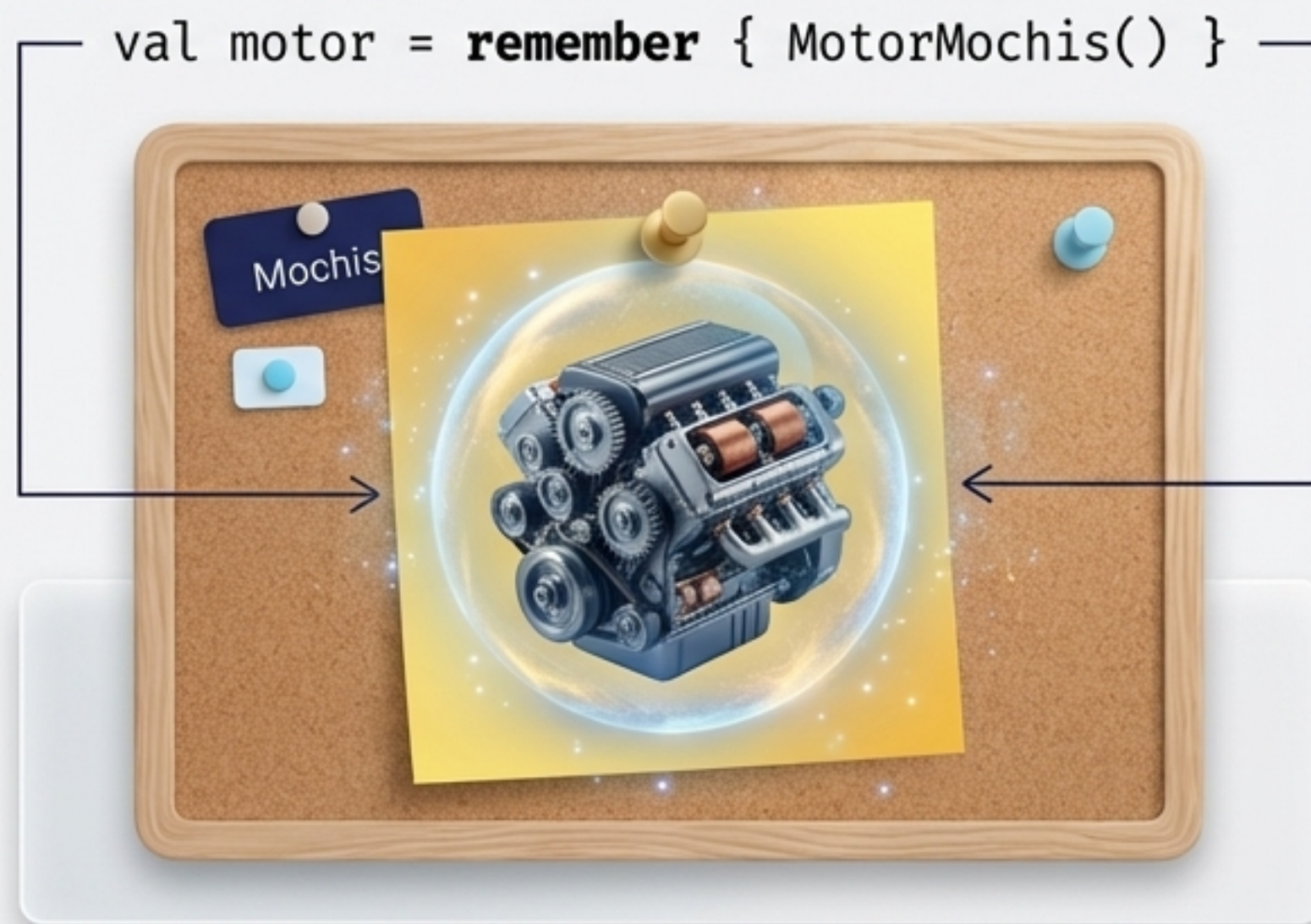


La Memoria de Compose: Entendiendo remember

En Compose, la pantalla se redibuja (recompone) docenas de veces por segundo. Si no guardamos nuestros datos, se borrarán en cada parpadeo.



Sin remember: El sistema crea y destruye el motor físico en cada fotograma. Un desastre.

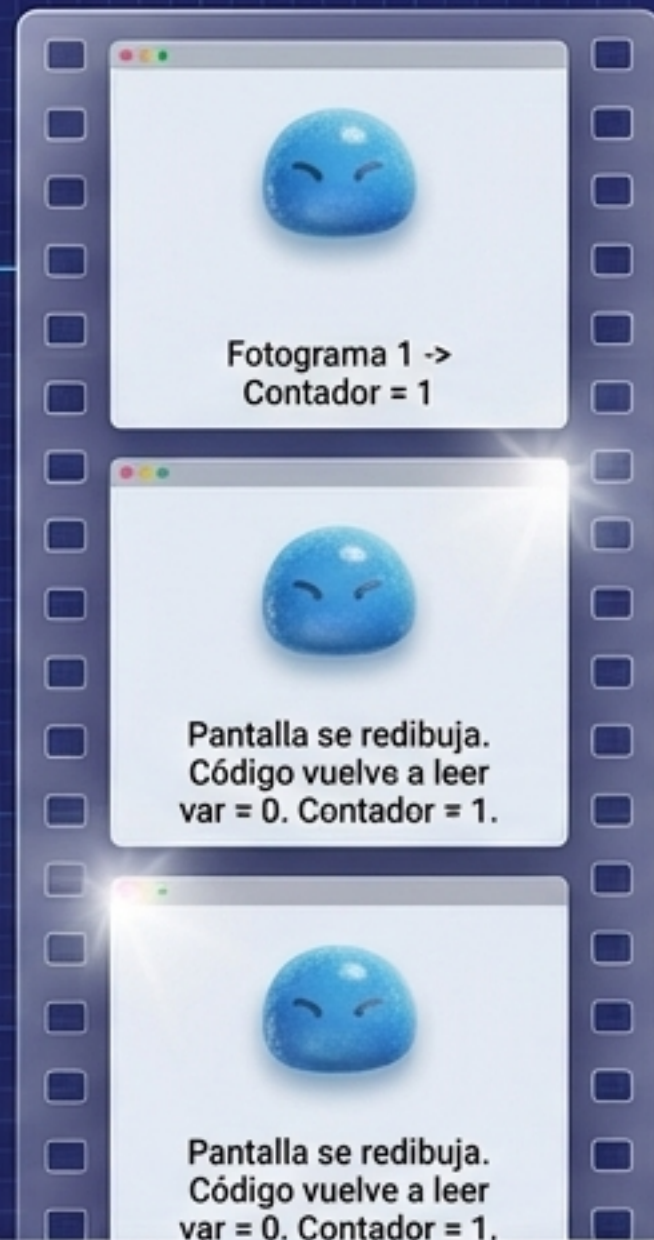


Con remember: Compose anota la existencia del motor y dice: 'Ya tengo uno, usaré el mismo en el siguiente redibujo'.

El Impacto de Olvidar la Memoria

Sin remember (El Bucle Infinito)

```
var contador =  
    mutableStateOf(0)
```



El tiempo nunca avanza.
El Mochi se queda
congelado en el aire.

Con remember (El Flujo Natural)

```
var contador by remember  
{ mutableStateOf(0)  
}
```



El tiempo fluye...
El Mochi cae en colte.



El tiempo fluye.
El Mochi cae con
gravedad perfecta.

Conciencia Espacial: Calculando los Límites

Para que un Mochi rebote, el motor debe saber dónde está el suelo y dónde están las paredes.

```
var tamanyoPantalla by remember {  
    mutableStateOf(IntSize.Zero)  
}
```

Comenzamos a ciegas.
Tamaño 0x0.

0x0

velocidadY =
-velocidadY

Modifier.onSizeChanged

En cuanto la pantalla se
dibuja por primera vez, este
sensor captura el ancho
y alto exacto en píxeles y
actualiza la variable.

El Toque Final: Dibujando en el Canvas



```
val medidorDeTexto = rememberTextMeasurer()
```



```
drawText(text = mochi.emoji,  
         topLeft = Offset(...))
```



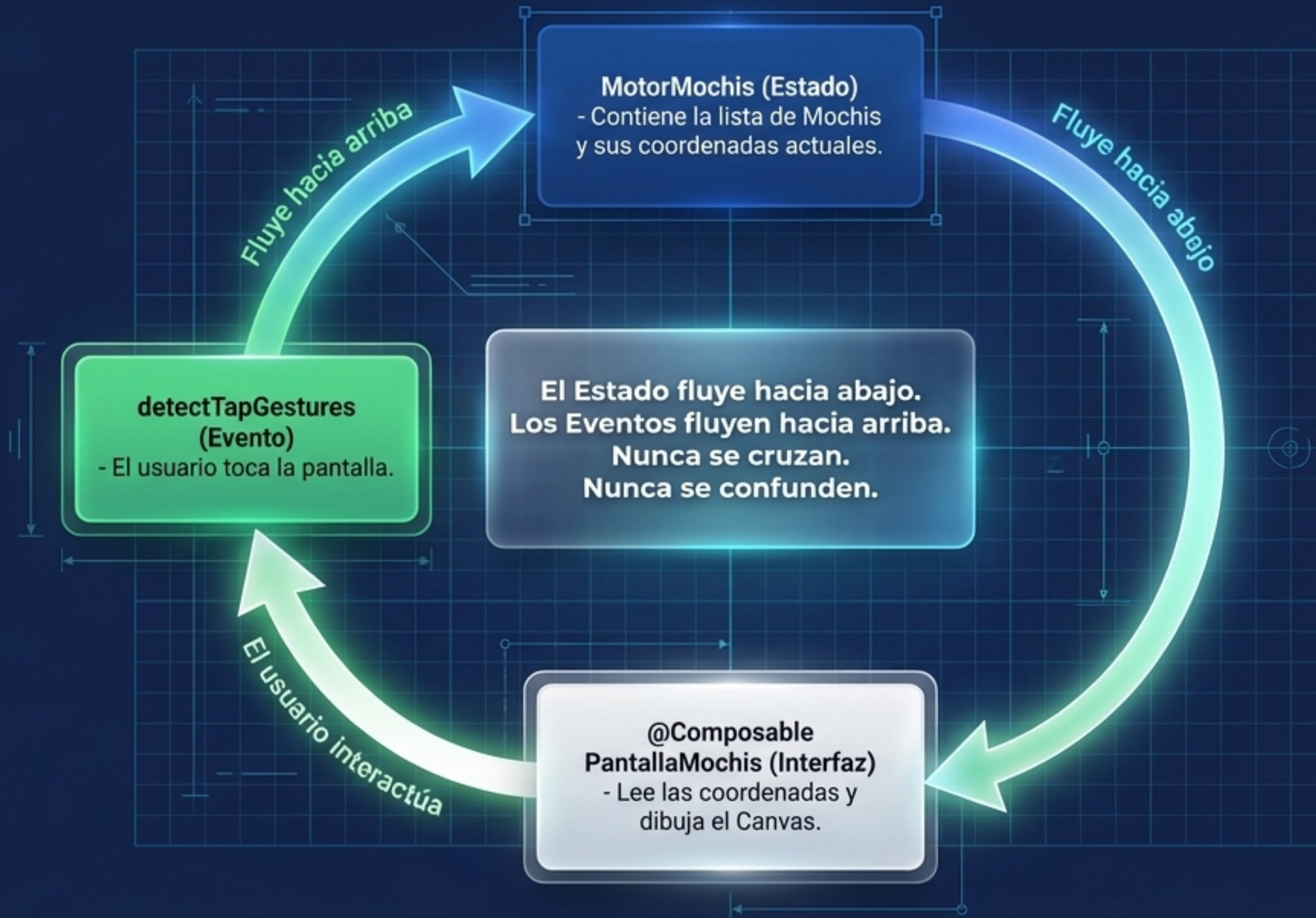
Los emojis no son imágenes libres; son tipografía (String). ¿Cómo centramos un texto exactamente en unas coordenadas X/Y?

El Medidor (TextMeasurer) calcula el ancho y alto exacto de la fuente de texto antes de pintar.

El Canvas utiliza la posición (X,Y) del Dato, ajusta el centro con el Medidor, e imprime el emoji.

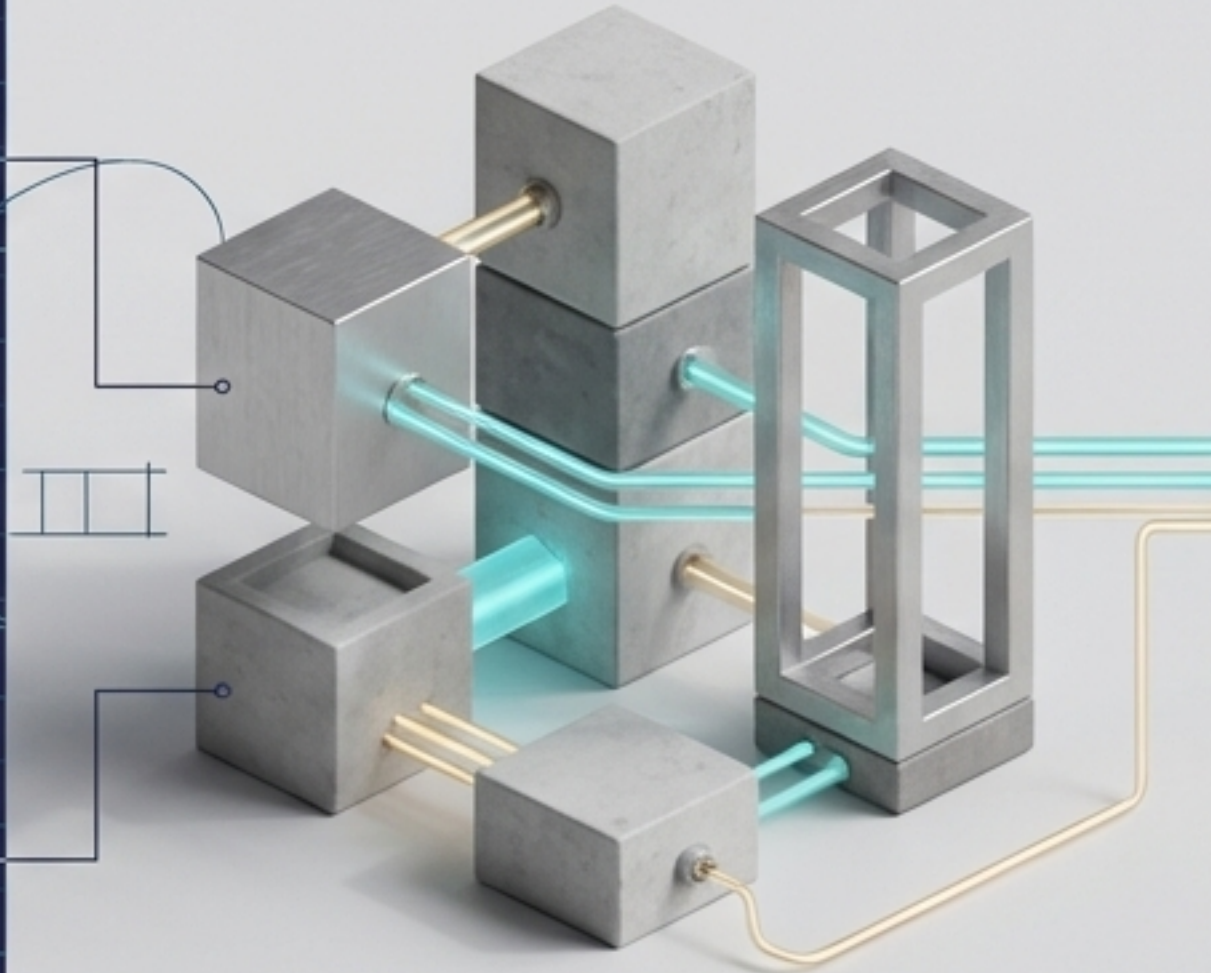
El Flujo de Datos Unidireccional (UDF)

El secreto detrás de todas las apps modernas de Android.

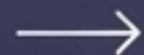


Tu Lienzo Está Vivo

```
data class Mochi(val x: Float,  
                val y: Float)  
  
val mochiList = remember {  
    mutableStateListOf<Mochi>()  
}  
  
val flow = remember {  
    ... datasource  
    flows: Data {  
        dibujaMochi.namea()  
        mutableStateSeat(nexn.,baaa)  
    }  
}
```



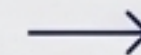
1. Estructura:
Tienes Datos
precisos (Mochi).



2. Memoria:
Tienes una Interfaz que
recuerda su estado
(remember).



3. Movimiento:
Tienes un flujo continuo
que dibuja, escucha y
recalcula (UDF).



Con solo 4 piezas, has
construido un universo físico.
Bienvenido a Jetpack Compose.

